In [1]:
```
import tensorflow as tf
import numpy as np
from tensorflow.keras.datasets import mnist
from tensorflow.keras.applications import VGG19
from tensorflow.keras.models import Model
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.layers import Input,GlobalAveragePooling2D, Dense
```

In [2]:
```
(x_train,y_train),(x_test,y_test) = mnist.load_data()
x_train.shape,y_train.shape
```

Out[2]:
```
((60000, 28, 28), (60000,))
```

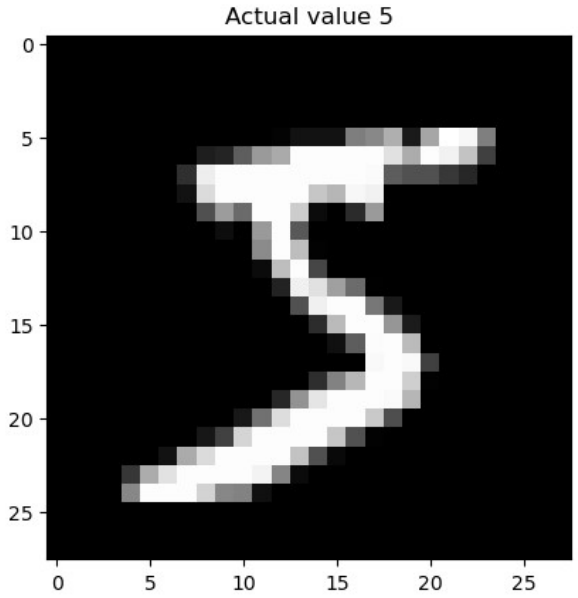
In [3]:
```
x_test.shape,y_test.shape
```

Out[3]:
```
((10000, 28, 28), (10000,))
```

In [4]:
```
import matplotlib.pyplot as plt
```

In [5]:
```
plt.imshow(x_train[0],cmap='gray')

plt.title(f'Actual value {y_train[0]}')
plt.show()
```



In [6]:
```
print(x_train.shape)
x_train = np.expand_dims(x_train,axis=-1)
x_test = np.expand_dims(x_test,axis=-1)
print(x_train.shape)
```

```
(60000, 28, 28)
(60000, 28, 28, 1)
```

In [7]:
```
print(x_train.shape)
x_train = np.repeat(x_train,3,axis = -1)
x_test = np.repeat(x_test,3,axis = -1)
print(x_train.shape)
```

```
(60000, 28, 28, 1)
(60000, 28, 28, 3)
```

In [8]:
```
print(x_train.shape)
x_train = tf.image.resize(x_train,(64,64))
x_test = tf.image.resize(x_test,(64,64))
print(x_train.shape)
```

```
(60000, 28, 28, 3)
(60000, 64, 64, 3)
```

In [9]:
```
y_train_cat = to_categorical(y_train,10)
y_test_cat = to_categorical(y_test,10)
```

In [10]:
```
base_model = VGG19(
    include_top=False,
    weights='imagenet',
    input_shape=(64, 64, 3),
)
```

In [11]:
```
base_model.trainable = False
```

In [12]:
```
inputs = Input(shape = (64,64,3))
x = base_model(inputs, training= False)
x = GlobalAveragePooling2D()(x)
x = Dense(128,activation='relu')(x)
output = Dense(10,activation="softmax")(x)
model = Model(inputs,output)
```

In [13]:
```
model.compile(optimizer='adam',loss = "categorical_crossentropy",metrics=['accur
```

In [14]:
```
model.summary()
```

Model: "functional"

Layer (type)	Output Shape
input_layer_1 (InputLayer)	(None, 64, 64, 3)
vgg19 (Functional)	(None, 2, 2, 512)
global_average_pooling2d (GlobalAveragePooling2D)	(None, 512)
dense (Dense)	(None, 128)
dense_1 (Dense)	(None, 10)

Total params: 20,091,338 (76.64 MB)

Trainable params: 66,954 (261.54 KB)

Non-trainable params: 20,024,384 (76.39 MB)

In [15]:
```
history=model.fit(x_train,y_train_cat,batch_size=2048,epochs=5,validation_split=
```

```
Epoch 1/5
27/27 ━━━━━━━━━━━━━━━━━━━━ 469s 17s/step - accuracy: 0.5185 - loss: 3.1386 - val_
accuracy: 0.8340 - val_loss: 0.6151
Epoch 2/5
27/27 ━━━━━━━━━━━━━━━━━━━━ 469s 17s/step - accuracy: 0.8667 - loss: 0.4827 - val_
accuracy: 0.9068 - val_loss: 0.3228
Epoch 3/5
27/27 ━━━━━━━━━━━━━━━━━━━━ 468s 17s/step - accuracy: 0.9094 - loss: 0.3114 - val_
accuracy: 0.9270 - val_loss: 0.2391
Epoch 4/5
27/27 ━━━━━━━━━━━━━━━━━━━━ 454s 16s/step - accuracy: 0.9271 - loss: 0.2423 - val_
accuracy: 0.9422 - val_loss: 0.1958
Epoch 5/5
27/27 ━━━━━━━━━━━━━━━━━━━━ 445s 16s/step - accuracy: 0.9387 - loss: 0.2038 - val_
accuracy: 0.9468 - val_loss: 0.1728
```

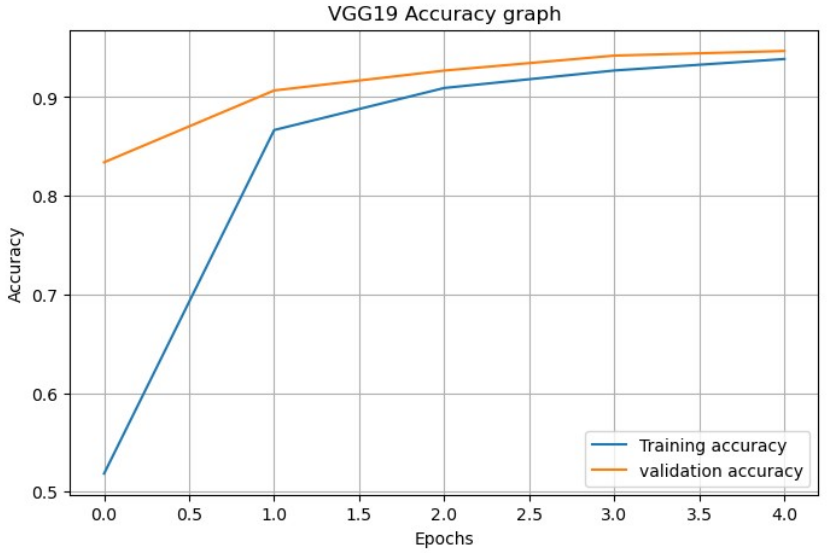
In [17]:
```
test_loss,test_accuracy = model.evaluate(x_test,y_test_cat)
test_loss,test_accuracy
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 73s 234ms/step - accuracy: 0.9412 - loss: 0.1996
```

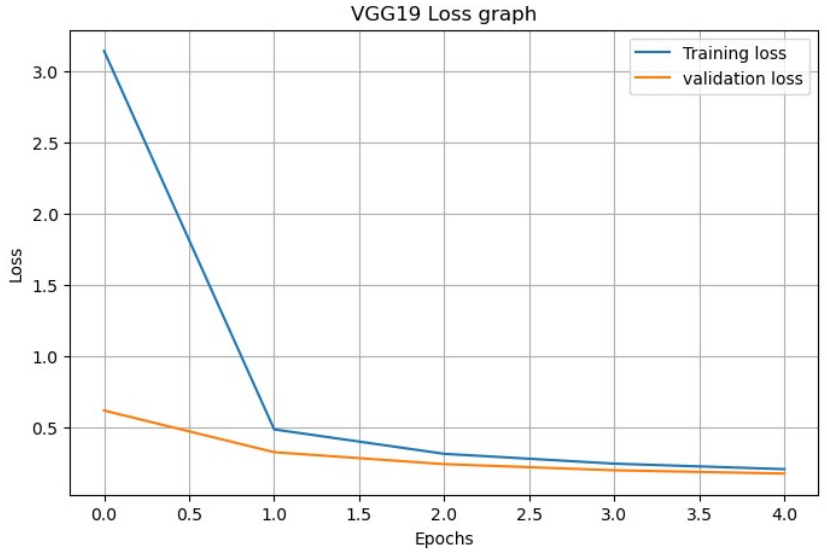
Out[17]:
```
(0.19964338839054108, 0.9412000179290771)
```

In [18]:
```
import matplotlib.pyplot as plt
```

In [20]:
```
plt.figure(figsize = (8,5))
plt.plot(history.history['accuracy'],label = "Training accuracy")
plt.plot(history.history['val_accuracy'],label = "validation accuracy")
plt.title("VGG19 Accuracy graph")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.grid()
plt.legend()
plt.show()
```



In [21]:
```
plt.figure(figsize = (8,5))
plt.plot(history.history['loss'],label = "Training loss")
plt.plot(history.history['val_loss'],label = "validation loss")
plt.title("VGG19 Loss graph")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.grid()
plt.legend()
plt.show()
```



In [22]:
```
predictions = model.predict(x_test[:10])
predicted_labels = np.argmax(predictions, axis=1)

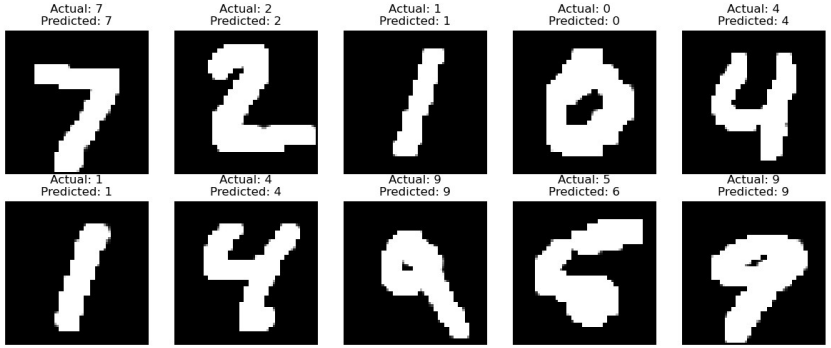
plt.figure(figsize=(12, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i])
    plt.title(
        f"Actual: {y_test[i]}\nPredicted: {predicted_labels[i]}"
    )
    plt.axis('off')

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 366ms/step
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..254.10547].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..254.5625].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..252.96973].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..254.375].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..252.96875].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..254.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..253.07422].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..254.8125].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..253.0].
Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [0.0..254.69629].
```



In [ ]: